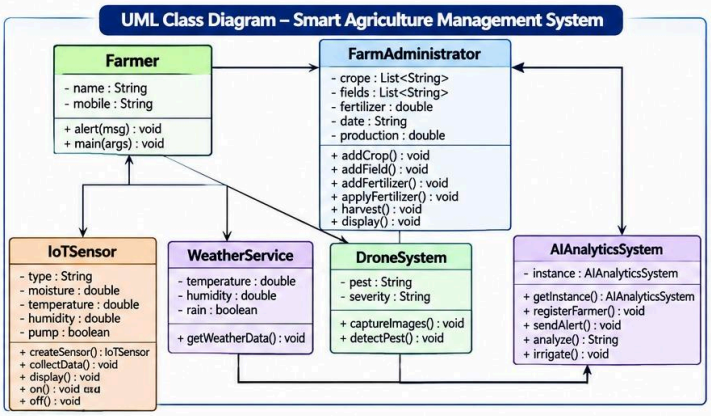
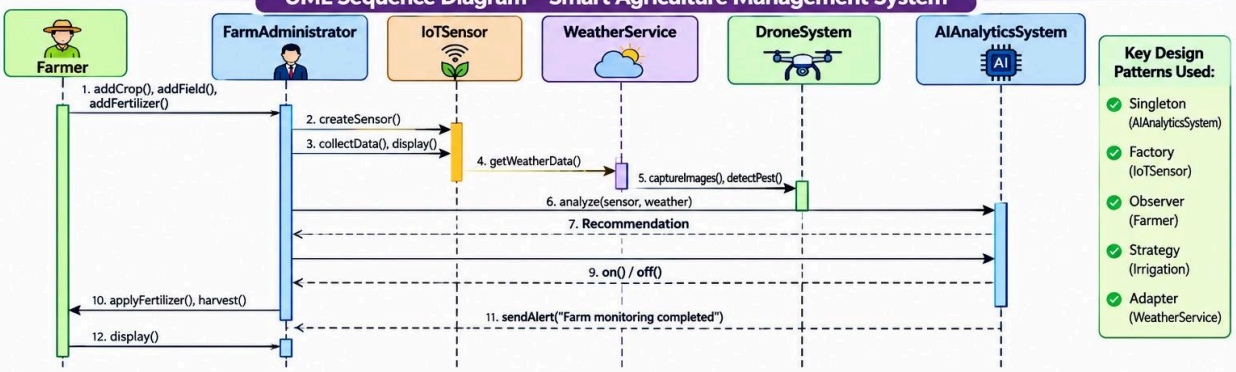
CRC Cards – Smart Agriculture Management System

Class	Responsibilities	Collaborators
Farmer	• Store name & mobile • Receive alerts • View farm data	FarmAdministrator, IoTSensor, WeatherService, DroneSystem, AIAnalyticsSystem
FarmAdministrator	• Add crops & fields • Manage fertilizer • Plan harvest	Farmer
IoTSensor	• Collect soil data • Control pump	AIAnalyticsSystem, Farmer
WeatherService	• Provide weather data	AIAnalyticsSystem
DroneSystem	• Capture images • Detect pests	Farmer
AIAnalyticsSystem	• Analyze data, send alerts, control irrigation	Farmer, IoTSensor, WeatherService

UML Class Diagram – Smart Agriculture Management System



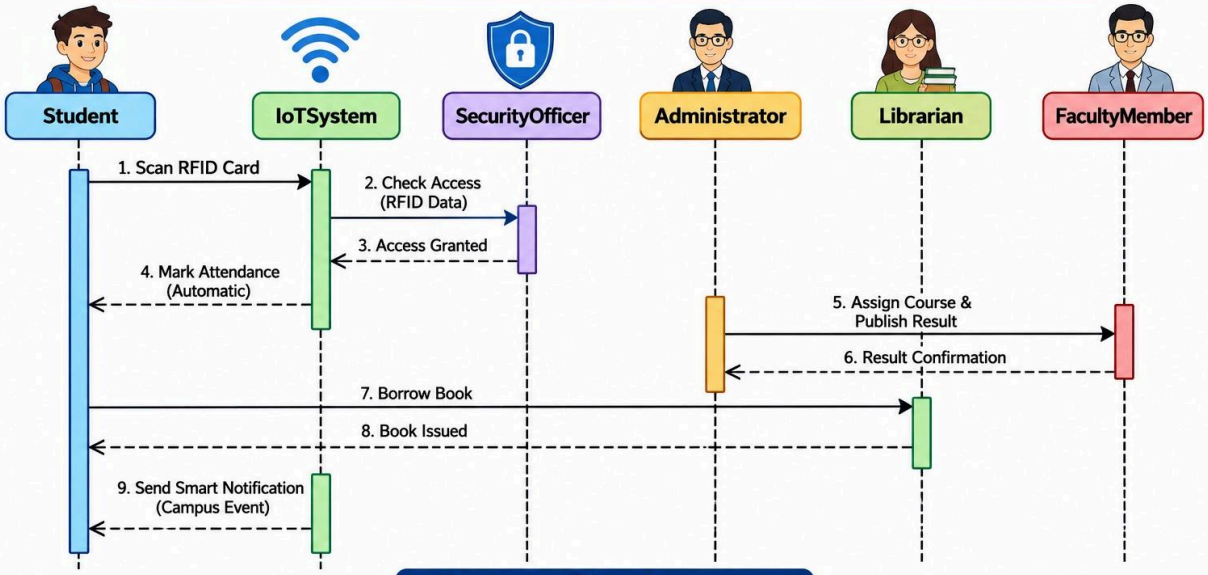
UML Sequence Diagram – Smart Agriculture Management System



Key Design Patterns Used:

- Singleton (AIAnalyticsSystem)
- Factory (IoTSensor)
- Observer (Farmer)
- Strategy (Irrigation)
- Adapter (WeatherService)

UML Sequence Diagram – Smart University Campus Management System (SUCMS)



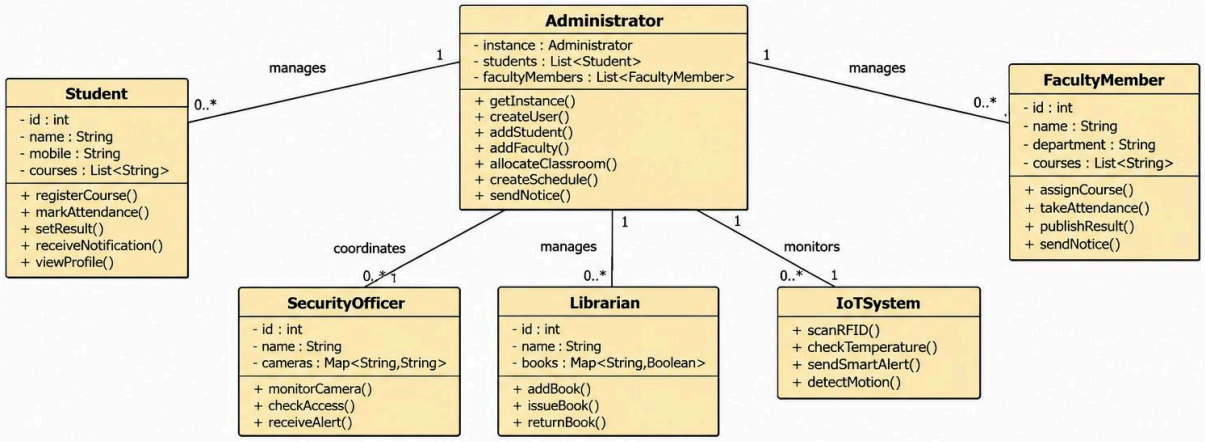
Sequence Flow Summary

1. Student scans RFID card at campus entry.
2. IoT system checks with Security Officer.
3. Access is granted and attendance is recorded.
4. Student gets confirmation.
5. Administrator assigns course and publishes result.
6. FacultyMember confirms the result.
7. Student borrows a book from Librarian.
8. Book is issued successfully.
9. Student receives smart notification.

CRC Card – Smart University Campus Management System (SUCMS)

Class	Responsibilities	Collaborators
Student	Register courses, manage profile, mark attendance, view results, receive notifications.	Administrator, FacultyMember, IoTSystem
FacultyMember	Assign courses, manage students, take attendance, publish results, send notices.	Administrator, Student
Administrator	Manage students and faculty, allocate classrooms, create schedules, manage university configuration.	Student, FacultyMember, SecurityOfficer, Librarian, IoTSystem
SecurityOfficer	Monitor CCTV cameras, check RFID access, receive security alerts.	Administrator, IoTSystem
Librarian	Add books, issue books, return books, adapt external records.	Administrator, Student
IoTSystem	Scan RFID cards, record attendance, monitor temperature, detect motion.	Student, SecurityOfficer

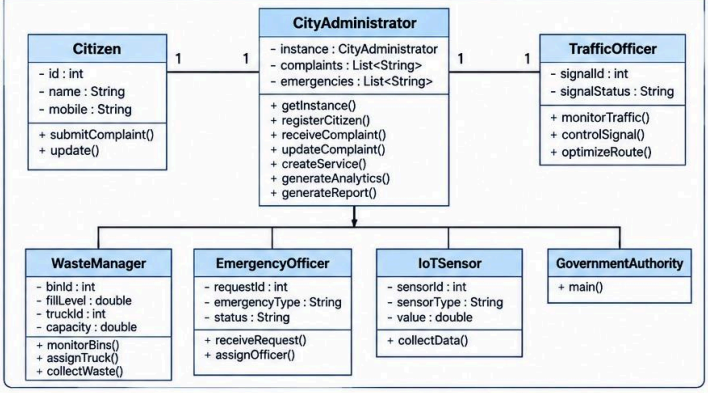
UML Class Diagram – SUCMS



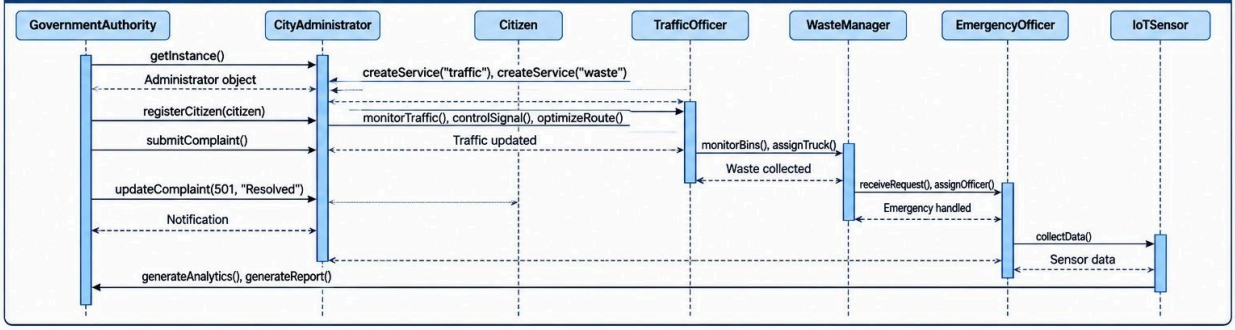
CRC Card – Dhaka Urban Management System (DUMS)

Class	Responsibilities	Collaborators
Citizen	Register, submit complaints, receive notifications	CityAdministrator
CityAdministrator	Manage citizens, complaints, services, analytics & reports	Citizen, WasteManager, EmergencyOfficer
TrafficOfficer	Monitor traffic, control signals, optimize routes	GovernmentAuthority
WasteManager	Monitor bins, assign trucks, collect waste	CityAdministrator
EmergencyOfficer	Receive emergency requests, assign officers	CityAdministrator
IoTSensor	Collect environmental data	GovernmentAuthority
GovernmentAuthority	Start & coordinate the system	All classes

UML Class Diagram – DUMS



UML Sequence Diagram – DUMS System Flow



PROJECT 1: DUMS (Dhaka Urban Management System)

PHASE 1: Requirement Analysis
1(a) Background & Three Problems:
*   **Background:** Dhaka city struggles with fragmented urban services, traffic congestion, and delayed emergency responses. The DUMS integrates these into a single platform to handle citizen complaints, dispatch services, and notify authorities.
*   **Problems:** 1. Fragmented services (citizens don't know where to report). 2. Delayed dispatch due to lack of real-time traffic data. 3. Poor communication (citizens are not updated on complaint status).
1(b) Primary & Specific Objectives:
*   **Primary:** Develop an OO subsystem that allows citizens to submit complaints, automatically dispatches the correct service using traffic data, and notifies stakeholders.
*   **Specific:** 1. Create specific services (Waste, Traffic, Emergency). 2. Integrate external traffic API. 3. Support different route strategies. 4. Notify observers on status changes.
1(c) Four Actors & System Boundary:
*   **Actors:** Citizen (submits complaints), City Administrator (manages workflow), Traffic Officer (monitors routes), IoT Sensor (environmental data).
*   **Boundary:** *Inside:* Repositories, Factories, RoutePlanners, Adapters, Publishers. *Outside:* Citizens, external Traffic API, physical sensors. (Note: `TrafficAPIAdapter` is internal; the actual API is external).
1(d) Functional & Non-Functional Requirements:
*   **FRs:** 1. Submit complaints with unique IDs (status: OPEN). 2. Create services via factory and reject unknown types. 3. Fetch traffic data via adapter and select route. 4. Change status to IN_PROGRESS only if a route is found; notify observers.
*   **NFRs:** 1. Maintainability (interchangeable route strategies). 2. Reliability (reject duplicate IDs; no status change if no route).

PHASE 2: Object Identification
2(a) Entity Classes:
1. `Citizen` (id, name)
2. `Complaint` (complaintId, status)
3. `WasteManager` (binId, fillLevel)
4. `TrafficOfficer` (signalId, status)
5. `EmergencyOfficer` (requestId, type)

2(b) Control Classes:
1. `CityAdministrator` (Coordinates workflow, updates status).
2. `UrbanServiceFactory` (Creates specific services).
3. `RoutePlanner` (Selects best route using strategy).

2(c) Boundary Classes:
1. `CitizenMobileApp` (Citizen interaction).

2. `TrafficAPIAdapter` (Translates external API data).
3. `AlertPublisher` (Sends notifications to dashboards).

PHASE 3: CRC Cards
1. **CityAdministrator (Control):** *Resp:* Receive complaints, trigger services, update status, publish alerts. *Collab:* Complaint, UrbanServiceFactory, RoutePlanner, AlertPublisher.
2. **Complaint (Entity):** *Resp:* Store ID and status (OPEN/IN_PROGRESS/RESOLVED); enforce valid transitions. *Collab:* CityAdministrator.
3. **UrbanServiceFactory (Control):** *Resp:* Create Waste/Traffic/Emergency services; reject unknowns. *Collab:* UrbanService (Interface).
4. **RoutePlanner (Control):** *Resp:* Hold Strategy; pick best route from candidates. *Collab:* RouteStrategy, TrafficAPIAdapter.
5. **TrafficAPIAdapter (Boundary):** *Resp:* Wrap external API; convert meters to km and seconds to minutes. *Collab:* ExternalTrafficAPI.
6. **AlertPublisher (Control):** *Resp:* Maintain observers; broadcast messages. *Collab:* Observer.

PHASE 4: UML Modeling
4(a) Class Diagram (Key Elements):
* *Classes:* `CityAdministrator` (Singleton), `Citizen`, `Complaint`, `UrbanService` (Interface), `WasteService`, `RoutePlanner`, `RouteStrategy` (Interface), `TrafficAPIAdapter`, `Observer` (Interface).
* *Relations:* `CityAdministrator` (1) —— (0..*) `Complaint` (Association). `CityAdministrator` ⇢ `UrbanServiceFactory` (Dependency). `RoutePlanner` ◇── `RouteStrategy` (Aggregation). `TrafficAPIAdapter` ◆── `ExternalTrafficAPI` (Composition). `CitizenNotifier` ⇢ `Observer` (Realization).

4(b) Sequence Diagram (Dispatch Response):
* *Flow:* Admin -> Complaint: `getComplaint()`. Admin -> Factory: `createService()`. Admin -> Adapter: `getRoutes()`. Admin -> Planner: `chooseRoute()`.
* *ALT Block:* `[Route Available]` -> Complaint: `setStatus(IN_PROGRESS)`, Publisher: `notify()`. `[No Route]` -> Complaint: `Status remains OPEN`.

PROJECT 2: AAS (Agricultural Automation System)

PHASE 1: Requirement Analysis
1(a) Background & Three Problems:
* **Background:** Farms rely on manual monitoring for irrigation and weather. The AAS automates this using sensors and weather data.
* **Problems:** 1. Delayed decisions. 2. Unnecessary water consumption. 3. Poor weather responsiveness.

1(b) Primary & Specific Objectives:
* **Primary:** Develop an OO system to monitor soil, get weather data via adapter, apply strategies, and notify farmers.
* **Specific:** 1. Monitor moisture. 2. Integrate weather API. 3. Automate pump control (Normal/Water-saving). 4. Send alerts.

1(c) Four Actors & System Boundary:
* **Actors:** Farmer, Farm Admin, IoT Sensor, Weather Service.
* **Boundary:** *Inside:* Sensors, Pump, Controllers, Adapters. *Outside:* Physical hardware, external Weather API.

1(d) Functional & Non-Functional Requirements:
* **FRs:** 1. Read moisture and reject values outside 0-100%. 2. Turn pump ON/OFF based on strategy. 3. Switch strategies at runtime. 4. Notify all observers of the decision.
* **NFRs:** 1. Maintainability (interchangeable strategies/adapters). 2. Reliability (invalid readings must never activate the pump).

PHASE 2: Object Identification
2(a) Entity Classes:
1. `SoilSensor` (sensorId, moisture)
2. `Pump` (pumpId, isOn)
3. `WeatherData` (temperature, rainExpected)
4. `FarmDataManager` (farmId, readings)
5. `Field` (fieldId, area)

2(b) Control Classes:
1. `IrrigationController` (Reads sensor, applies strategy, controls pump).
2. `SensorFactory` (Creates sensors based on type).
3. `AlertPublisher` (Manages subscriptions and notifications).

2(c) Boundary Classes:
1. `WeatherAdapter` (Converts external string data to boolean).
2. `FarmerNotifier` (Delivers alerts to the farmer).
3. `FarmDashboard` (Displays system status).

PHASE 3: CRC Cards
1. **IrrigationController (Control):** *Resp:* Read moisture, validate, apply strategy, control pump, publish. *Collab:* SoilSensor, Strategy, WeatherAdapter, Pump, AlertPublisher.
2. **SoilSensor (Entity):** *Resp:* Provide moisture value; allow updates. *Collab:* IrrigationController.
3. **SensorFactory (Control):** *Resp:* Instantiate SOIL/TEMP sensors; reject unknowns. *Collab:* Sensor (Interface).
4. **WeatherAdapter (Boundary):** *Resp:* Wrap external API; convert String to boolean. *Collab:* ExternalWeatherAPI.

5. **Strategy (Control):** *Resp:* Define rules for Normal and Water-Saving. *Collab:* IrrigationController.
6. **AlertPublisher (Control):** *Resp:* Maintain observers; broadcast messages. *Collab:* Observer.

PHASE 4: UML Modeling
4(a) Class Diagram (Key Elements):
* *Classes:* `SoilSensor`, `IrrigationPump`, `IrrigationController`, `SensorFactory`, `WeatherAdapter`, `AlertPublisher`, `IrrigationStrategy` (Interface), `Observer` (Interface).
* *Relations:* `SoilSensor` ⇢ `Sensor` (Realization). `IrrigationController` ◇── `IrrigationStrategy` (Aggregation). `WeatherAdapter` ◆── `ExternalWeatherAPI` (Composition). `AlertPublisher` (1) ─── (0..*) `Observer` (Association).

4(b) Sequence Diagram (Irrigation Check):
* *Flow:* Farmer -> Controller: `requestCheck()`. Controller -> Sensor: `read()`. Controller -> WeatherAdapter: `isRainExpected()`. Controller -> Strategy: `irrigate()`.
* *ALT Block:* `[Irrigation Required]` -> Controller -> Pump: `turnOn()`, Publisher: `publish()` -> Farmer: `update()`. `[Not Required]` -> Pump: `turnOff()`, Publisher: `publish()` -> Farmer: `update()`.

PROJECT 3: SUCMS (Smart University Campus Management System)

PHASE 1: Requirement Analysis
1(a) Background & Three Problems:
* **Background:** Universities use fragmented systems for registration, grading, and library access, leading to inefficiencies.
* **Problems:** 1. Inconsistent registration checks (credit limits exceeded). 2. Manual, error-prone score calculations. 3. Poor notification targeting (personal results broadcast to all).

1(b) Primary & Specific Objectives:
* **Primary:** Develop an OO system that validates registrations, computes scores via policies, sends targeted notices, and integrates external library data.
* **Specific:** 1. Create accounts via factory. 2. Enforce 6-credit limit. 3. Calculate Theory/Lab scores. 4. Adapter integration for library.

1(c) Four Actors & System Boundary:
* **Actors:** Student, Faculty, Admin, Librarian.
* **Boundary:** *Inside:* `CourseRegistrationManager`, `EvaluationService`, `NoticePublisher`, `LibraryAdapter`. *Outside:* Users, external Library system.

1(d) Functional & Non-Functional Requirements:

*   **FRs:** 1. Register students only if below 6 credits and course not full. 2. Validate marks (0-100) and compute using selected strategy. 3. Broadcast public notices; filter personal results by ID. 4. Query external library via adapter.
*   **NFRs:** 1. Maintainability (interchangeable strategies/adapters). 2. Reliability (rejected registration leaves credits unchanged; invalid marks stop evaluation).

PHASE 2: Object Identification
2(a) Entity Classes:
1. `Student` (id, name)
2. `Course` (code, credits, capacity)
3. `Marks` (exam, coursework, project)
4. `LibraryBook` (isbn, copies)
5. `Notice` (noticeId, recipientId)

2(b) Control Classes:
1. `CourseRegistrationManager` (Enforces capacity, duplicate, credit limits).
2. `EvaluationService` (Calculates scores via Strategy).
3. `AccountFactory` (Creates Student/Faculty/Admin roles).

2(c) Boundary Classes:
1. `StudentPortal` (Displays notices and personal results).
2. `FacultyPortal` (Inputs marks, views public notices).
3. `LibraryAdapter` (Translates external library text to boolean availability).

PHASE 3: CRC Cards
1. **CourseRegistrationManager (Control):** *Resp:* Check duplicates, capacity, 6-credit limit; update records. *Collab:* Student, Course, UniversityConfig.
2. **EvaluationService (Control):** *Resp:* Hold active strategy; calculate score; trigger publication. *Collab:* EvaluationStrategy, Marks, NoticePublisher.
3. **Student (Entity):** *Resp:* Retain identity; own registrations. *Collab:* UserAccount, CourseRegistrationManager.
4. **NoticePublisher (Control):** *Resp:* Maintain observers; broadcast tagged messages. *Collab:* CampusObserver, StudentPortal, FacultyPortal.
5. **LibraryAdapter (Boundary):** *Resp:* Query external system; convert data to boolean. *Collab:* LibraryService, ExternalLibrarySystem.
6. **UniversityConfig (Entity/Singleton):** *Resp:* Provide single instance; enforce 6-credit max. *Collab:* CourseRegistrationManager.

PHASE 4: UML Modeling
4(a) Class Diagram (Key Elements):
*   *Classes:* `UniversityConfig` (Singleton), `AccountFactory`, `CourseRegistrationManager`, `EvaluationService`, `NoticePublisher`, `LibraryAdapter`, `UserAccount` (Interface), `EvaluationStrategy` (Interface), `CampusObserver` (Interface).

*   *Relations:* `AccountFactory` ⇢ `UserAccount` (Dependency). `EvaluationService` ◇── `EvaluationStrategy` (Aggregation). `LibraryAdapter` ◆── `ExternalLibrarySystem` (Composition). `NoticePublisher` (1) ─── (0..*) `CampusObserver` (Association).

4(b) Sequence Diagram (Publish Result):
*   *Flow:* Faculty -> Marks: `new Marks()`. Faculty -> EvaluationService: `publishResult()`.
*   *ALT Block:* `[Valid Marks]` -> EvaluationService: `strategy.evaluate()`, Publisher: `publish(studentId, score)` -> StudentPortal: `update()` (filters by ID). `[Invalid Marks]` -> Marks: `throw Exception` (stops process).

AAS:

```
import java.util.*;

// 1. SENSOR
interface Sensor {
    String type();
    double read();
}

class SoilMoistureSensor implements Sensor {
    Scanner sc;

    SoilMoistureSensor(Scanner sc) {
        this.sc = sc;
    }

    public String type() {
        return "Soil Moisture Sensor";
    }

    public double read() {
        System.out.print("Enter soil moisture (%): ");
        return sc.nextDouble();
    }
}

class TemperatureSensor implements Sensor {
    Scanner sc;

    TemperatureSensor(Scanner sc) {
        this.sc = sc;
    }

    public String type() {
        return "Temperature Sensor";
    }

    public double read() {
        System.out.print("Enter temperature: ");
        return sc.nextDouble();
    }
}

// 2. FACTORY PATTERN
class SensorFactory {
    Scanner sc;

    SensorFactory(Scanner sc) {
        this.sc = sc;
```

```
    }

    Sensor createSensor(int choice) {
        if (choice == 1)
            return new SoilMoistureSensor(sc);
        else if (choice == 2)
            return new TemperatureSensor(sc);
        else
            return null;
    }
}

// 3. WEATHER ADAPTER PATTERN
class WeatherData {
    boolean rainExpected;

    WeatherData(boolean rain) {
        rainExpected = rain;
    }
}

interface WeatherService {
    WeatherData current();
}

class ExternalWeatherAPI {
    boolean getRainForecast(boolean rain) {
        return rain;
    }
}

class WeatherAPIAdapter implements WeatherService {
    ExternalWeatherAPI api;
    boolean rain;

    WeatherAPIAdapter(ExternalWeatherAPI api, boolean rain) {
        this.api = api;
        this.rain = rain;
    }

    public WeatherData current() {
        return new WeatherData(api.getRainForecast(rain));
    }
}

// 4. STRATEGY PATTERN
interface IrrigationStrategy {
    boolean shouldIrrigate(double moisture, WeatherData weather);
}

class NormalIrrigationStrategy implements IrrigationStrategy {
    public boolean shouldIrrigate(double moisture, WeatherData weather) {
        return moisture < 40 && !weather.rainExpected;
    }
}

class WaterSavingStrategy implements IrrigationStrategy {
    public boolean shouldIrrigate(double moisture, WeatherData weather) {
        return moisture < 25 && !weather.rainExpected;
    }
}

// 5. IRRIGATION PUMP
class IrrigationPump {
    void setOn(boolean on) {
        System.out.println(on ? "Pump is ON" : "Pump is OFF");
    }
}

// 6. OBSERVER PATTERN
```

```
interface AlertObserver {
    void update(String message);
}

class FarmerNotifier implements AlertObserver {
    public void update(String message) {
        System.out.println("Farmer: " + message);
    }
}

class FarmDashboard implements AlertObserver {
    public void update(String message) {
        System.out.println("Dashboard: " + message);
    }
}

class AlertPublisher {
    List<AlertObserver> observers = new ArrayList<>();

    void subscribe(AlertObserver o) {
        observers.add(o);
    }

    void publish(String message) {
        for (AlertObserver o : observers)
            o.update(message);
    }
}

// 7. FIELD AND FARM DATA MANAGER
class Field {
    String name;

    Field(String name) {
        this.name = name;
    }
}

class FarmDataManager {
    List<Field> fields = new ArrayList<>();

    void addField(Field f) {
        fields.add(f);
        System.out.println("Field added successfully!");
    }

    void showFields() {
        if (fields.isEmpty()) {
            System.out.println("No fields available.");
            return;
        }

        for (Field f : fields)
            System.out.println("Field: " + f.name);
    }
}

// 8. IRRIGATION CONTROLLER
class IrrigationController {
    IrrigationStrategy strategy;
    WeatherService weather;
    IrrigationPump pump;
    AlertPublisher alerts;

    IrrigationController(IrrigationStrategy s, WeatherService w,
                         IrrigationPump p, AlertPublisher a) {
        strategy = s;
        weather = w;
        pump = p;
        alerts = a;
```

```
    }

    void check(Sensor sensor) {
        double moisture = sensor.read();
        WeatherData data = weather.current();

        System.out.println("\nSensor: " + sensor.type());
        System.out.println("Reading: " + moisture);
        System.out.println("Rain Expected: " + data.rainExpected);

        boolean result = strategy.shouldIrrigate(moisture, data);

        pump.setOn(result);

        if (result)
            alerts.publish("Irrigation Started!");
        else
            alerts.publish("Irrigation Not Required!");
    }
}

// 9. MAIN CLASS
public class AASExam {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        SensorFactory factory = new SensorFactory(sc);
        ExternalWeatherAPI api = new ExternalWeatherAPI();
        IrrigationPump pump = new IrrigationPump();
        AlertPublisher publisher = new AlertPublisher();
        FarmDataManager manager = new FarmDataManager();

        // Automatically register observers
        publisher.subscribe(new FarmerNotifier());
        publisher.subscribe(new FarmDashboard());

        IrrigationStrategy strategy = new NormalIrrigationStrategy();

        while (true) {
            System.out.println("\n===== SMART IRRIGATION SYSTEM =====");
            System.out.println("1. Add Farm Field");
            System.out.println("2. View Farm Fields");
            System.out.println("3. Select Irrigation Strategy");
            System.out.println("4. Check Irrigation");
            System.out.println("5. Exit");

            System.out.print("Enter choice: ");
            int choice = sc.nextInt();
            sc.nextLine();

            switch (choice) {

                case 1:
                    System.out.print("Enter field name: ");
                    String name = sc.nextLine();
                    manager.addField(new Field(name));
                    break;

                case 2:
                    manager.showFields();
                    break;

                case 3:
                    System.out.println("1. Normal Strategy");
                    System.out.println("2. Water Saving Strategy");
                    System.out.print("Choose strategy: ");

                    int s = sc.nextInt();
```

```
                if (s == 1) {
                    strategy = new NormalIrrigationStrategy();
                    System.out.println("Normal Strategy Selected!");
                }
                else if (s == 2) {
                    strategy = new WaterSavingStrategy();
                    System.out.println("Water Saving Strategy Selected!");
                }
                else {
                    System.out.println("Invalid choice!");
                }
                break;

            case 4:
                System.out.println("1. Soil Moisture Sensor");
                System.out.println("2. Temperature Sensor");
                System.out.print("Choose sensor: ");

                Sensor sensor = factory.createSensor(sc.nextInt());

                if (sensor == null) {
                    System.out.println("Invalid sensor choice!");
                    break;
                }

                System.out.print("Rain expected? (1=Yes, 0=No): ");
                int rain = sc.nextInt();

                if (rain != 0 && rain != 1) {
                    System.out.println("Invalid weather choice!");
                    break;
                }

                WeatherService weather =
                    new WeatherAPIAdapter(api, rain == 1);

                IrrigationController controller =
                    new IrrigationController(strategy, weather, pump, publisher);

                controller.check(sensor);
                break;

            case 5:
                System.out.println("Thank you for using Smart Irrigation System!");
                sc.close();
                return;

            default:
                System.out.println("Invalid choice!");
        }
    }
  }
}
```

DUMS:

```
import java.util.*;

// Complaint Class
class Complaint {
    int id;
    String citizen, type, location, status;

    Complaint(int id, String citizen, String type, String location) {
```

```
        this.id = id;
        this.citizen = citizen;
        this.type = type;
        this.location = location;
        status = "Pending";
    }

    void display() {
        System.out.println("ID: " + id + " | Citizen: " + citizen
                + " | Type: " + type + " | Location: " + location
                + " | Status: " + status);
    }
}

// 1. Repository Pattern
class UrbanDatabase {
    ArrayList<Complaint> complaints = new ArrayList<>();

    void addComplaint(Complaint c) {
        complaints.add(c);
    }

    Complaint findComplaint(int id) {
        for (Complaint c : complaints) {
            if (c.id == id)
                return c;
        }
        return null;
    }

    void showAll() {
        for (Complaint c : complaints)
            c.display();
    }

    void showCitizen(String name) {
        for (Complaint c : complaints) {
            if (c.citizen.equalsIgnoreCase(name))
                c.display();
        }
    }
}

// 2. Factory Pattern
interface UrbanService {
    void provideService();
}

class WasteService implements UrbanService {
    public void provideService() {
        System.out.println("Waste service selected.");
    }
}

class TrafficService implements UrbanService {
    public void provideService() {
        System.out.println("Traffic service selected.");
    }
}

class EmergencyService implements UrbanService {
```

```
    public void provideService() {
        System.out.println("Emergency service selected.");
    }
}

class UrbanServiceFactory {
    static UrbanService create(String type) {
        if (type.equalsIgnoreCase("waste"))
            return new WasteService();
        if (type.equalsIgnoreCase("traffic"))
            return new TrafficService();
        if (type.equalsIgnoreCase("emergency"))
            return new EmergencyService();

        return null;
    }
}

// 3. Observer Pattern
interface ComplaintObserver {
    void notifyUser(String message);
}

class CitizenNotifier implements ComplaintObserver {
    public void notifyUser(String message) {
        System.out.println("Citizen Notification: " + message);
    }
}

class GovernmentDashboard implements ComplaintObserver {
    public void notifyUser(String message) {
        System.out.println("Government Dashboard: " + message);
    }
}

class StatusPublisher {
    ArrayList<ComplaintObserver> observers = new ArrayList<>();

    void addObserver(ComplaintObserver o) {
        observers.add(o);
    }

    void notifyAllUsers(String message) {
        for (ComplaintObserver o : observers)
            o.notifyUser(message);
    }
}

// 4. Strategy Pattern
interface RouteStrategy {
    void findRoute();
}

class ShortestRoute implements RouteStrategy {
    public void findRoute() {
        System.out.println("Shortest Route: Road A");
        System.out.println("Distance: 5 km | Estimated Time: 20 minutes");
    }
}

class FastestRoute implements RouteStrategy {
```

```
    public void findRoute() {
        System.out.println("Fastest Route: Road B");
        System.out.println("Distance: 8 km | Estimated Time: 12 minutes");
    }
}

class RoutePlanner {
    RouteStrategy strategy;

    void setStrategy(RouteStrategy s) {
        strategy = s;
    }

    void planRoute() {
        strategy.findRoute();
    }
}

// 5. Adapter Pattern
class ExternalTrafficAPI {
    String getTrafficData() {
        return "Road A: Low Traffic, Road B: High Traffic, Road C: Medium Traffic";
    }
}

interface TrafficServiceAPI {
    void showTrafficData();
}

class TrafficAPIAdapter implements TrafficServiceAPI {
    ExternalTrafficAPI api = new ExternalTrafficAPI();

    public void showTrafficData() {
        System.out.println("\nTraffic Information:");
        System.out.println(api.getTrafficData());
    }
}

// Citizen Role
class Citizen {
    void menu(UrbanDatabase db, StatusPublisher publisher) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        int choice;

        do {
            System.out.println("\n--- Citizen Menu ---");
            System.out.println("1. Submit Complaint");
            System.out.println("2. View My Complaints");
            System.out.println("3. Exit");
            System.out.print("Choice: ");
            choice = sc.nextInt();
            sc.nextLine();

            if (choice == 1) {
                System.out.print("Complaint ID: ");
                int id = sc.nextInt();
                sc.nextLine();
```

```
            if (db.findComplaint(id) != null) {
                System.out.println("Complaint ID already exists.");
                continue;
            }

            System.out.print("Service (waste/traffic/emergency): ");
            String type = sc.nextLine();

            UrbanService service = UrbanServiceFactory.create(type);

            if (service == null) {
                System.out.println("Invalid service.");
                continue;
            }

            service.provideService();

            System.out.print("Location: ");
            String location = sc.nextLine();

            db.addComplaint(new Complaint(id, name, type, location));
            publisher.notifyAllUsers("New complaint submitted. ID: " + id);
        }

        else if (choice == 2) {
            db.showCitizen(name);
        }

    } while (choice != 3);
  }
}

// Administrator Role
class Administrator {
    void menu(UrbanDatabase db, StatusPublisher publisher) {
        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            System.out.println("\n--- Administrator Menu ---");
            System.out.println("1. View All Complaints");
            System.out.println("2. Update Complaint Status");
            System.out.println("3. Plan Route");
            System.out.println("4. View Traffic Data");
            System.out.println("5. Exit");
            System.out.print("Choice: ");
            choice = sc.nextInt();

            if (choice == 1) {
                db.showAll();
            }

            else if (choice == 2) {
                System.out.print("Enter Complaint ID: ");
                int id = sc.nextInt();

                Complaint c = db.findComplaint(id);

                if (c != null) {
                    c.status = "Resolved";
```

```
                System.out.println("Complaint status updated to Resolved.");
                publisher.notifyAllUsers("Complaint " + id + " is now Resolved.");
            } else {
                System.out.println("Complaint not found.");
            }
        }

        else if (choice == 3) {
            System.out.println("1. Shortest Route");
            System.out.println("2. Fastest Route");
            System.out.print("Choose route: ");
            int option = sc.nextInt();

            RoutePlanner planner = new RoutePlanner();

            if (option == 1)
                planner.setStrategy(new ShortestRoute());
            else if (option == 2)
                planner.setStrategy(new FastestRoute());
            else {
                System.out.println("Invalid choice.");
                continue;
            }

            planner.planRoute();
        }

        else if (choice == 4) {
            TrafficServiceAPI traffic = new TrafficAPIAdapter();
            traffic.showTrafficData();
        }

    } while (choice != 5);
  }
}

// Main Class
public class DUMS {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        UrbanDatabase db = new UrbanDatabase();

        StatusPublisher publisher = new StatusPublisher();
        publisher.addObserver(new CitizenNotifier());
        publisher.addObserver(new GovernmentDashboard());

        while (true) {
            System.out.println("\n===== DUMS =====");
            System.out.println("1. Citizen");
            System.out.println("2. Administrator");
            System.out.println("3. Exit");
            System.out.print("Select Role: ");

            int choice = sc.nextInt();
            sc.nextLine();

            if (choice == 1)
                new Citizen().menu(db, publisher);

            else if (choice == 2)
```

```
            new Administrator().menu(db, publisher);

        else if (choice == 3) {
            System.out.println("Program ended.");
            break;
        }

        else
            System.out.println("Invalid choice.");
    }
  }
}
```

SUCMS:

```
import java.util.*;

// SINGLETON PATTERN
class UniversityConfig {
  static UniversityConfig obj = new UniversityConfig();
  int maxCredit = 20;

  private UniversityConfig() {}

  static UniversityConfig getInstance() {
    return obj;
  }
}

// FACTORY PATTERN
interface UserAccount {
  String getId();
}

class Student implements UserAccount {
  String id, name;
  ArrayList<Course> myCourses = new ArrayList<>();
  ArrayList<String> notifications = new ArrayList<>();

  Student(String id, String name) {
    this.id = id;
    this.name = name;
  }

  public String getId() {
    return id;
  }
}

class Faculty implements UserAccount {
  String id, name;

  Faculty(String id, String name) {
    this.id = id;
    this.name = name;
  }

  public String getId() {
    return id;
  }
}
```

```
class AccountFactory {
    static UserAccount create(String role, String id, String name) {
        if (role.equals("student"))
            return new Student(id, name);
        return new Faculty(id, name);
    }
}

// COURSE AND REGISTRATION
class Course {
    String code, name;
    int credit, capacity;
    ArrayList<String> students = new ArrayList<>();

    Course(String code, String name, int credit, int capacity) {
        this.code = code;
        this.name = name;
        this.credit = credit;
        this.capacity = capacity;
    }
}

class RegistrationManager {
    static ArrayList<Course> courses = new ArrayList<>();

    static void viewCourses() {
        for (Course c : courses)
            System.out.println(c.code + " - " + c.name +
                    " | Credit: " + c.credit);
    }

    static void register(Student s, String code) {
        for (Course c : courses) {
            if (c.code.equalsIgnoreCase(code)) {
                int total = 0;
                for (Course x : s.myCourses)
                    total += x.credit;

                if (c.students.contains(s.id))
                    System.out.println("Already registered!");
                else if (c.students.size() >= c.capacity)
                    System.out.println("Course is full!");
                else if (total + c.credit > UniversityConfig.getInstance().maxCredit)
                    System.out.println("Credit limit exceeded!");
                else {
                    c.students.add(s.id);
                    s.myCourses.add(c);
                    System.out.println("Registration successful!");
                }
                return;
            }
        }
        System.out.println("Course not found!");
    }
}

// OBSERVER PATTERN
interface CampusObserver {
    void update(String message);
}

class StudentPortal implements CampusObserver {
    Student student;

    StudentPortal(Student s) {
        student = s;
    }

    public void update(String message) {
        student.notifications.add(message);
```

```
    }
}

class NoticePublisher {
    ArrayList<CampusObserver> observers = new ArrayList<>();

    void addObserver(CampusObserver o) {
        observers.add(o);
    }

    void publish(String message) {
        for (CampusObserver o : observers)
            o.update(message);

        System.out.println("Notification published!");
    }
}

// STRATEGY PATTERN
class Marks {
    double obtained, total;

    Marks(double obtained, double total) {
        this.obtained = obtained;
        this.total = total;
    }
}

interface EvaluationStrategy {
    double calculate(Marks m);
}

class TheoryEvaluation implements EvaluationStrategy {
    public double calculate(Marks m) {
        return (m.obtained / m.total) * 100;
    }
}

class LabEvaluation implements EvaluationStrategy {
    public double calculate(Marks m) {
        return (m.obtained / m.total) * 100;
    }
}

class EvaluationService {
    EvaluationStrategy strategy;

    void setStrategy(EvaluationStrategy s) {
        strategy = s;
    }

    double publishResult(Marks m) {
        return strategy.calculate(m);
    }
}

// ADAPTER PATTERN
class ExternalLibrarySystem {
    String getInfo() {
        return "Total Books: 500\nAvailable Books: 350\nBorrowed Books: 150";
    }
}

interface LibraryService {
    String getInformation();
}

class LibraryAdapter implements LibraryService {
    ExternalLibrarySystem library = new ExternalLibrarySystem();
```

```
    public String getInformation() {
        return library.getInfo();
    }
}

// MAIN CLASS SUCMS
public class SUCMS {
    static Scanner sc = new Scanner(System.in);

    static HashMap<String, Student> students = new HashMap<>();
    static HashMap<String, Faculty> faculties = new HashMap<>();

    static NoticePublisher publisher = new NoticePublisher();

    // STUDENT REGISTRATION
    static void studentRegistration() {
        System.out.print("Enter ID: ");
        String id = sc.nextLine();

        if (students.containsKey(id)) {
            System.out.println("ID already exists!");
            return;
        }

        System.out.print("Enter Name: ");
        String name = sc.nextLine();

        Student s = (Student) AccountFactory.create("student", id, name);
        students.put(id, s);
        publisher.addObserver(new StudentPortal(s));

        System.out.println("Registration successful!");
    }

    // FACULTY REGISTRATION
    static void facultyRegistration() {
        System.out.print("Enter ID: ");
        String id = sc.nextLine();

        if (faculties.containsKey(id)) {
            System.out.println("ID already exists!");
            return;
        }

        System.out.print("Enter Name: ");
        String name = sc.nextLine();

        Faculty f = (Faculty) AccountFactory.create("faculty", id, name);
        faculties.put(id, f);

        System.out.println("Registration successful!");
    }

    // STUDENT MENU
    static void studentUI(Student s) {
        while (true) {
            System.out.println("\n--- STUDENT MENU ---");
            System.out.println("1. View Courses");
            System.out.println("2. Register Course");
            System.out.println("3. My Courses and Total Credit");
            System.out.println("4. Notifications");
            System.out.println("5. Logout");

            int ch = Integer.parseInt(sc.nextLine());

            if (ch == 1) {
                RegistrationManager.viewCourses();
            }
            else if (ch == 2) {
                System.out.print("Enter Course Code: ");
```

```
            RegistrationManager.register(s, sc.nextLine());
        }
        else if (ch == 3) {
            int total = 0;
            for (Course c : s.myCourses) {
                System.out.println(c.code + " - " + c.name +
                        " | Credit: " + c.credit);
                total += c.credit;
            }
            System.out.println("Total Credit: " + total);
        }
        else if (ch == 4) {
            if (s.notifications.isEmpty())
                System.out.println("No notifications!");
            else
                for (String n : s.notifications)
                    System.out.println(n);
        }
        else if (ch == 5) {
            break;
        }
    }
}

// FACULTY MENU
static void facultyUI(Faculty f) {
    while (true) {
        System.out.println("\n--- FACULTY MENU ---");
        System.out.println("1. View Students");
        System.out.println("2. Publish Result");
        System.out.println("3. Logout");

        int ch = Integer.parseInt(sc.nextLine());

        if (ch == 1) {
            for (Student s : students.values())
                System.out.println(s.id + " - " + s.name);
        }
        else if (ch == 2) {
            System.out.print("Enter Student ID: ");
            Student s = students.get(sc.nextLine());

            if (s == null) {
                System.out.println("Student not found!");
                continue;
            }

            System.out.print("Obtained Marks: ");
            double obtained = Double.parseDouble(sc.nextLine());

            System.out.print("Total Marks: ");
            double total = Double.parseDouble(sc.nextLine());

            if (total <= 0 || obtained < 0 || obtained > total) {
                System.out.println("Invalid marks!");
                continue;
            }

            System.out.println("1. Theory Evaluation");
            System.out.println("2. Lab Evaluation");
            int type = Integer.parseInt(sc.nextLine());

            EvaluationService service = new EvaluationService();

            if (type == 1)
                service.setStrategy(new TheoryEvaluation());
            else if (type == 2)
                service.setStrategy(new LabEvaluation());
            else {
                System.out.println("Invalid choice!");
```

```
                continue;
            }

            double result = service.publishResult(
                    new Marks(obtained, total));

            publisher.publish("Your result is: " + result + "%");
            System.out.println("Result: " + result + "%");
        }
        else if (ch == 3) {
            break;
        }
    }
}

// ADMIN MENU
static void adminUI() {
    while (true) {
        System.out.println("\n--- ADMIN MENU ---");
        System.out.println("1. View Courses");
        System.out.println("2. Add Course");
        System.out.println("3. View Registrations");
        System.out.println("4. Library Information");
        System.out.println("5. Publish Notice");
        System.out.println("6. Back");

        int ch = Integer.parseInt(sc.nextLine());

        if (ch == 1) {
            RegistrationManager.viewCourses();
        }
        else if (ch == 2) {
            System.out.print("Course Code: ");
            String code = sc.nextLine();

            System.out.print("Course Name: ");
            String name = sc.nextLine();

            System.out.print("Credit: ");
            int credit = Integer.parseInt(sc.nextLine());

            System.out.print("Capacity: ");
            int capacity = Integer.parseInt(sc.nextLine());

            boolean exists = false;
            for (Course c : RegistrationManager.courses)
                if (c.code.equalsIgnoreCase(code))
                    exists = true;

            if (exists || credit <= 0 || capacity <= 0) {
                System.out.println("Invalid or duplicate course!");
            } else {
                RegistrationManager.courses.add(
                        new Course(code, name, credit, capacity));
                System.out.println("Course added!");
            }
        }
        else if (ch == 3) {
            for (Course c : RegistrationManager.courses) {
                System.out.println("\n" + c.code + " - " + c.name);
                for (String id : c.students)
                    System.out.println("Student ID: " + id);
            }
        }
        else if (ch == 4) {
            LibraryService library = new LibraryAdapter();
            System.out.println(library.getInformation());
        }
        else if (ch == 5) {
            System.out.print("Enter Notice: ");
```

```
            publisher.publish("Notice: " + sc.nextLine());
        }
        else if (ch == 6) {
            break;
        }
    }
}

// MAIN MENU
public static void main(String[] args) {
    RegistrationManager.courses.add(
            new Course("CSE101", "Programming", 3, 40));
    RegistrationManager.courses.add(
            new Course("CSE102", "Database", 3, 30));

    while (true) {
        System.out.println("\n===== UNIVERSITY SYSTEM =====");
        System.out.println("1. Student");
        System.out.println("2. Faculty");
        System.out.println("3. Administrator");
        System.out.println("4. Exit");

        int ch = Integer.parseInt(sc.nextLine());

        if (ch == 1) {
            System.out.println("\n1. Registration");
            System.out.println("2. Login");
            int option = Integer.parseInt(sc.nextLine());

            if (option == 1)
                studentRegistration();
            else if (option == 2) {
                System.out.print("Enter Student ID: ");
                Student s = students.get(sc.nextLine());

                if (s != null)
                    studentUI(s);
                else
                    System.out.println("Student not found! Register first.");
            }
        }
        else if (ch == 2) {
            System.out.println("\n1. Registration");
            System.out.println("2. Login");
            int option = Integer.parseInt(sc.nextLine());

            if (option == 1)
                facultyRegistration();
            else if (option == 2) {
                System.out.print("Enter Faculty ID: ");
                Faculty f = faculties.get(sc.nextLine());

                if (f != null)
                    facultyUI(f);
                else
                    System.out.println("Faculty not found! Register first.");
            }
        }
        else if (ch == 3) {
            adminUI();
        }
        else if (ch == 4) {
            System.out.println("Goodbye!");
            break;
        }
    }
}
}
```